



COMILLAS
UNIVERSIDAD PONTIFICIA

ICAI

ICADE

CIHS

Desarrollo de aplicaciones y servicio

Documentación sprint 1 - NaiveBay

Sergio Jiménez Romero

Pablo Tuñón Laguna

1. Introducción

El Proyecto *NaiveBay* es una página de subastas con diseño iterativo que se encuentra almacenada en el controlador de versiones *GitHub* y se despliega en un dominio gratuito de la plataforma *Vercel*. Tal y como se ha comentado, el proyecto es iterativo y está organizado a través de la metodología scrum, es por ello que cada iteración se organiza como un sprint con sus correspondientes pasos. Objetivos del sprint 1:

- Creación de página de inicio de sesión de usuario.
- Creación de página de registro de nuevos usuarios.
- Creación de página de detalles de subasta.
- Creación de página de resultados de búsqueda.
- Creación de página principal.

La creación de dichas páginas implica lo siguiente:

- Desarrollo del HTML y CSS necesarios para la estructuración de la página.
- Desarrollo del JavaScript para que funcione la parte de front end de la aplicación, es decir:
 - o Correcta redirección entre páginas con sus correspondientes botones.
 - o Verificaciones en la creación de usuarios.
 - o Verificaciones en el acceso a la web.
 - o Una barra lateral desplegable con espacio para añadir futura información.
 - o Un carrusel de imágenes para visualizar distintas imágenes de un producto (sólo funciona para el ejemplo contenido en `bidding_details_example.html`).

A lo largo del siguiente escrito se detallan las implementaciones realizadas así como las asignaciones realizadas.

2. Implementaciones

A lo largo de la siguiente sección se detallan brevemente los ficheros empleados para alcanzar los objetivos propuestos. Para completar el trabajo se han desarrollado un total de 24 ficheros:

2.1 CSS

Existen 7 ficheros del tipo Cascading Style Sheets una para dar formatos específicos a cada una de las 6 páginas disponibles y uno más para algunos criterios generales. Cabe destacar que también hay algunas estructuras que se han repetido pero que no son comunes para todas las páginas. La nomenclatura de las clases dentro del css se ha tratado de que siga la metodología BEM haciendo más clara su lectura y edición. Entre múltiples propiedades se han usado las siguientes:

- Hover para cuando el ratón pase por encima de un componente.
- Paddings y margins para proporcionar distintos tipos de espaciados.
- Background color y color para modificar los colores.
- Border radius, height y width para modificar la apariencia de los bloques que forman la página web.
- Flex para editar elementos dentro de un bloque de la página.
- Grid para construir la propia estructura de bloques y clases dentro de la página.
- Unidades relativas así como porcentajes para facilitar la integración en distintos dispositivos.

2.2 HTML

Se han desarrollado un total de 6 páginas con estructura html:

- bidding_details: página que recoge el funcionamiento y detalles de una subasta concreta. Fue desarrollada en conjunción con dummyjson para poder mostrar el correcto funcionamiento de la página web ya que no se iban a hacer tantos ejemplos o subastas a mano como en el caso del posterior fichero.
- bidding_details_example: página ejemplo estructurada para ser similar al mock up, mostrar toda la información detallada así como precisa del item (como será en un futuro). También sirve para mostrar el carrusel de imágenes que en el futuro podrán usar todas las subastas.
- login_page: página que permite a un usuario registrado iniciar sesión, aunque como todavía no tiene conectada ninguna base de datos no puede más que denegar la entrada a los usuarios y redirigir a los usuarios a otras páginas.
- register_page: similar a la página de inicio de sesión, actualmente no tiene todas las funcionalidades que tendrá en el futuro. Posteriormente en la parte de java script se explicarán las validaciones que esta realiza.

- `search_results`: es una página que contiene los resultados de hacer búsquedas a través de la barra de búsquedas, está conectada con los ejemplos que genera `bidding_details`.
- `index`: la página principal de todo NaiveBay, muestra subastas con gran actividad así como otras que finalizarán en breve.

Cabe destacar que todas las pestañas tienen los botones de registro de usuario, inicio de sesión, al pulsar el logo te dirigen a la página principal. Las pestañas también cuentan con una barra lateral todavía sin una utilidad clara y una barra de búsqueda funcional para buscar subastas. Además se ha tratado de que las páginas hagan uso de un HTML semántico y estructurado para facilitar la lectura y la modularización.

2.3 JS

El java script desarrollado se encarga de proporcionarle interacción al usuario, para lo cual se han desarrollado 9 ficheros:

- `bidding_page`: Encargado de rellenar la plantilla de detalles de subasta con ejemplos aunque sin hacerlo de forma detallada, sólo añade una foto, nombre y precio, lo demás es heredado del ejemplo del reloj Hamilton.
- `carrusel`: en la página de ejemplo de detalle de subasta, toma las imágenes disponibles de un contenedor, muestra una como predeterminada (la primera) y se encarga de sustituir dicha imagen por otras vistas del modelo en caso de que el usuario las seleccione.
- `login_button_state`: permite una redirección a las páginas de registro de usuario o de inicio de sesión sin emplear hipervínculos de html. Si se pincha el icono de la página también es posible ir a la página principal.
- `main_page_suggestions`: añade la funcionalidad de ver los ítems con más pujas y cuyas pujas finalizan en breve de la página principal.
- `password_validation`: permite validar que el usuario introducido está metiendo la contraseña correcta en la pestaña de inicio de sesión, se puede probar con el usuario *test* y la contraseña *test*.
- `register_validation`: también una validación, pero algo más extensa que la anterior ya que se encarga de verificar:
 - Ninguna de las contraseñas es nula (ni la inicial introducida por el usuario ni la confirmación) y ambas son idénticas.
 - Las contraseñas tienen al menos un carácter en mayúsculas, minúsculas, especial, numérico y tiene una extensión mínima de 8 caracteres (verificado a través de expresiones regulares).
 - El correo introducido no está vacío y contiene @ además de . al menos una vez.
 - Se ha seleccionado una ciudad y una comunidad autónoma.
 - El cumpleaños no es en una fecha que todavía no ha sucedido.

- No se trata de una verificación, pero es una validación js presente en el documento ya que está relacionada. Se trata de modificar el desplegable de ciudad en función de del la clave de comunidad autónoma que se seleccione.
- search_functions: le aporta funcionalidad a la barra de búsqueda, permitiendo que busque entre las páginas presentes en el directivo de páginas, si la encuentra la mostrará por pantalla, en caso contrario llamará al posterior fichero.
- search_page_results: gestiona la visualización de resultados de búsqueda de productos en la página web. Obtiene los productos de una API y permite ordenar los resultados por precio y relevancia mostrando los tres primeros de cada categoría.
- sidebar_activation: Le aporta a la barra lateral (que todavía no tiene contenidos) la posibilidad de desplegarse al pulsar las 3 rayas y ocultarse al pulsar la cruz.

3. Metodología SCRUM

El sprint fue estructurado siguiendo la metodología ágil SCRUM aunque con algunas modificaciones:

- Las historias de usuario se redactaron en base a las proporcionadas y fueron almacenadas en un proyecto de GitHub enlazado al repositorio de implementaciones.
- Es cierto que no se hizo un *product backlog* un *selected product backlog* o un *sprint backlog* ya que el equipo estaba limitado por los objetivos del propio sprint, los cuales ya estaban definidos al inicio del sprint. Estas historias se asignaron tratando no sólo de que todos los miembros realizaran implementaciones de los tres tipos posibles (HTML, CSS, JS) sino que el trabajo fuera lo más equitativo posible. Dado que no se conocían de antemano la complejidad de las implementaciones, tampoco se usaron story points.
- Se realizaron dos *daily scrums* durante el fin de semana por organización y un problema con una pull request que se comentará en la *sprint review*.
- Al finalizar el sprint, la implementación de NaiveBay cuenta con una serie de nuevas funcionalidades:
 - Páginas de registro de usuario, inicio de sesión, detalles de subasta, principal, resultado de búsqueda desarrolladas.
 - Desarrollado el JS para comunicar todas las páginas entre sí, validar inputs, realizar búsquedas y ver un carrusel de imágenes en los detalles de una subasta.
 - CSS para organizar y visualizar las distintas páginas implementadas.
- Al realizar una retrospectiva algo superficial (por el momento) de los resultados del sprint así como los problemas afrontados, se identificaron dos campos de mejora:
 - El equipo debe tener una mayor comunicación especialmente en el caso de elementos comunes para no obstaculizar otras implementaciones y no repetir código. Esto se observó principalmente cuando ambos miembros desarrollaron un header para almacenar botones y barras de búsqueda y los elementos posteriormente no cuadraban.
 - Se deben hacer pull requests a la rama develop desde las ramas de implementación de forma más habitual. Se estaban haciendo cada vez que se terminaba una implementación, sin embargo, por este motivo al intentar hacer el merge surgieron conflictos que llevaron 3h de resolución.

4. Reparto de implementaciones

Las tareas se trataron de repartir entre los integrantes del grupo de tal manera que todos tuvieran la oportunidad de trabajar en todos los ámbitos de desarrollo abiertos. Se dividieron las páginas a implementar y cada uno desarrolló tanto el mockup, html, css y js presente:

- Sergio Jiménez: página de inicio de sesión, página principal y página de resultado de subastas (incluida la interacción con la API para obtener los campos que generalizan subastas).
- Pablo Tuñón: página de registro de usuario, página de detalles de subasta ejemplificada e integración de página genérica de detalles de subasta con los resultados del js que llama a la API.